CSC311: Introduction to Machine Learning

Project Proposal

Which Sport Is It? Predicting a Sport from Survey Answers

Submitted by

Group [Number]

Ethan Yang, Sixuan Song, Yuzhou Wu, Zhihan Fan

University of Toronto
Fall 2026

1 Data Exploration

Task and how the test set was set aside. Each row is one survey response about a sport, and the label is one of three classes (*Baseball*, *Basketball*, *Soccer*). The file has 870 rows from 290 respondents, and each respondent answered once for every sport. *Before looking at any feature*, we split the respondents with a fixed seed (`split_data.py`) into 202 for training (606 rows), 44 for validation (132 rows) and 44 for testing (132 rows). Splitting *by respondent ID* keeps a person's three answers together, so a model cannot profit from recognising someone's writing style, which would not carry over to the TA/instructor test set. The ID itself is an arbitrary index and is not used as a feature. **Everything below is computed on the 606 training rows only** (`explore.py` never reads the other files). The test file is opened only once, for the final accuracy estimate. Every split is exactly one third per class (no class imbalance).

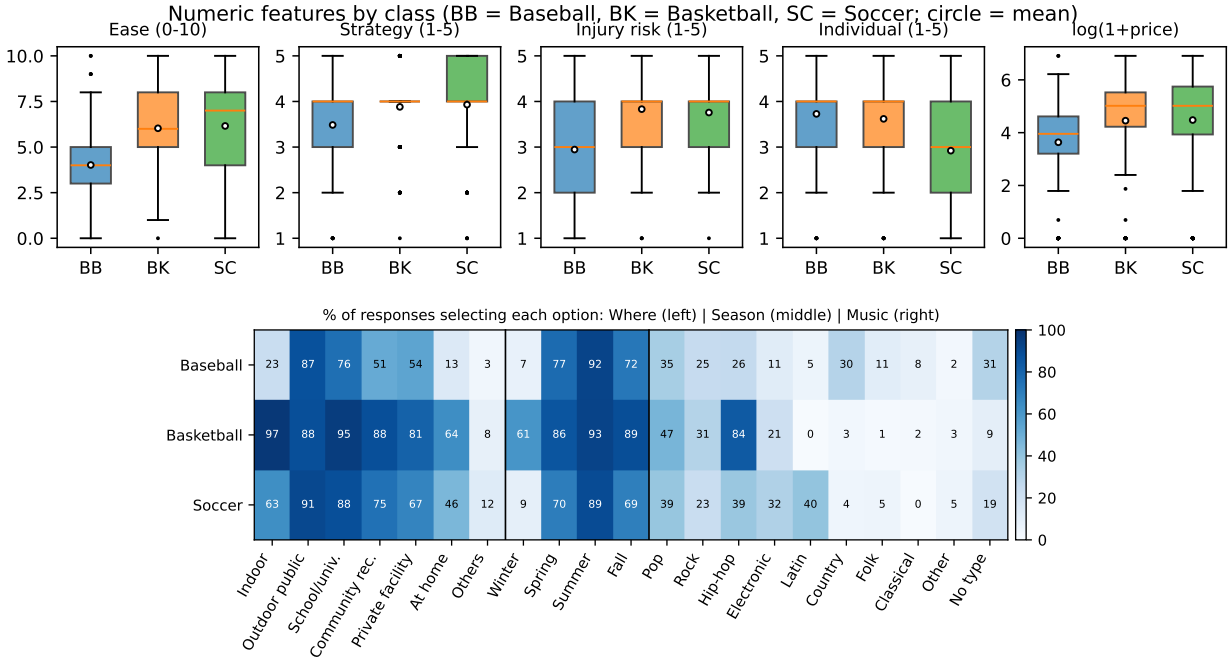


Figure 1: Training split only. Top: numeric features by class. Bottom: % of each class's responses selecting each multi-select option.

Feature types and distributions (Table 1). *Ease* is roughly symmetric on 0–10 (mean 5.4, SD 2.4). The Likert items lean towards *Agree*, and their pairwise correlations are at most 0.31, so no pair is redundant. *Price* is very right-skewed (skewness 6.7, median $100, maximum $5000, 13% answered $0). Free-text answers are short (median 1–2 words), so the encoding is sparse: 581 features for 606 rows (minimum word frequency 2), and 417 of the 564 binary features occur in under 1% of rows. Two options are nearly constant across classes (*Outdoor public space* 87–91%, *Summer* 89–93%), so they carry little information.

Missing values, outliers and inconsistencies. All forms of empty answer (NaN, empty string, "n/a", "-") are first mapped to one missing marker. Missing rates are low, but *Song*

Table 1: Feature types, missing rate (training split) and encoding.

Feature	Type	Missing	Encoding
Ease of starting (0–10)	numeric	0%	median-impute, standardise
Strategy, Injury risk, Individual	Likert (5 levels)	0%	ordinal 1–5, standardise
Where, Season, Music	multi-select	0%	multi-hot (7, 4, 10 indicators)
Food, Colour, Song, Feeling	free text	2.0, 0.8, 7.8, 0.0%	binary bag-of-words per question
Price willing to pay	free text → $	0.7% + 2.3% unparseable	cap $1000, $\log(1+x)$, standardise

is missing for 10.9% of Baseball and 9.9% of Basketball rows versus 2.5% of Soccer rows. Missingness is therefore informative, so we add a 0/1 *missing* indicator for each question and impute numeric features with the training median. Answers such as "none" are real answers and stay as words. Text varies in case, spacing and plurals ("Hotdog", "hot dogs") and contains placeholders ("[this sport]"), so our tokenizer lower-cases, drops placeholders and stop words, joins "hot dog" and strips plural "s". Price answers come in many formats ("$1,000", "200CAD", "100-200", "Nothing"). We parse the number, take the midpoint of ranges and map refusals to $0. The 2.3% of answers with no number ("very willing") are treated as missing. For outliers, the IQR rule flags 12% of prices (above $425), too many to call errors, so we cap price at $1000 (8 answers) and take $\log(1+x)$, which reduces the skewness from 6.7 to −1.1. All other values are in range.

Preprocessing and why. *Likert* items are ordinal 1–5 rather than one-hot. This keeps their order in one feature instead of five, which matters when there are about as many features as rows. *Numeric* features are standardised, $x' = (x-\mu)/\sigma$, with μ, σ from the training rows. Gradient descent, L2 penalties and neural networks are scale-sensitive. *Multi-select* answers become multi-hot 0/1 vectors, since several options can be chosen. *Free text* becomes a binary bag-of-words with a separate vocabulary for each question. We use presence rather than counts because answers are 1–2 words long. Words seen fewer than `min_df` times in training are dropped, and words first seen at prediction time are ignored. `min_df` and binary vs. TF-IDF weighting are tuned (Section 2). Every learned quantity (medians, μ, σ, vocabularies) is fitted on training data only (`preprocess.py`).

Which features are predictive. We compare the gap between class means with the within-class SD, and compute each feature's mutual information (MI) with the label. *Ease* separates Baseball (mean 4.0) from Basketball/Soccer (6.0/6.2), with gap/SD = 1.0. Baseball also scores lower on *injury risk* (2.95 vs. 3.8), and Soccer scores lower on *individual brilliance* (2.9 vs. 3.6). *Strategy* and *price* are weak (gap/SD < 0.5). In Fig. 1, several options pick out one class: *Indoor* (23/97/63% for Baseball/Basketball/Soccer), *Winter* (61% for Basketball, under 10% otherwise), *Hip-hop* (84% Basketball), *Latin* (40% Soccer) and *Country* (30% Baseball). The strongest single features are words used almost only for one class: "waka"/"shakira" (100% Soccer), "take me out to the ball game" (≥95% Baseball), "green" (95% Soccer), "orange" (95% Basketball) and "peanut" (93% Baseball). The highest MI is for `song:waka`, `where=Indoor` and `colour:green` (0.31–0.32 bits). **Implications:** many sparse indicators, each adding some evidence for one class, suit a regularised linear model; threshold effects and interactions (indoor *and* winter) suit tree ensembles; with $p \approx n$, all models need strong regularisation.

2 Planned Methodology

Model families. Each family is motivated by Section 1, and each can be re-implemented with numpy alone in `pred.py`.

1. **Softmax (multinomial logistic) regression with L2.** Most features are sparse indicators, and each adds a small, roughly independent amount of evidence for one class. That is exactly the additive log-odds model that softmax regression assumes. It has the lowest variance of our candidates, which matters with 581 features and 606 rows, and its weights are interpretable.
2. **Random forest** [1]. Several effects are threshold-shaped (Baseball if ease ≤ 4, Basketball if *Winter*) and may interact. Trees capture both without scaling and are unaffected by the remaining skew. Averaging many decorrelated trees removes most of a single tree's variance.
3. **Multilayer perceptron (MLP).** Hidden units can learn non-linear combinations of features (e.g. a word that matters only together with a season). With L2 regularisation and early stopping, it is a flexible middle ground between the two models above.

We decided not to use k-nearest neighbours: in 581 mostly-zero dimensions, distances are dominated by irrelevant word features (the curse of dimensionality), and untuned kNN was clearly worse in our preliminary check (Table 3). We also ruled out fine-tuning a language model, because 606 rows are too few and `pred.py` may only use numpy and pandas.

Training setup. Let $\mathbf{x}^{(i)} \in \mathbb{R}^D$ be an encoded response with label $y^{(i)} \in \{1, 2, 3\}$. Models are trained with scikit-learn 1.9 [2] on top of our own `preprocess.py`, and every random seed is fixed to 311.

- *Softmax regression:* $\mathbf{z} = \mathbf{W}\mathbf{x}+\mathbf{b}$ and $p(y = k \mid \mathbf{x}) = e^{z_k}/\sum_j e^{z_j}$. We minimise $\frac{1}{N}\sum_i -\log p(y^{(i)} \mid \mathbf{x}^{(i)}) + \frac{\lambda}{2}\|\mathbf{W}\|_F^2$, where $C = 1/(\lambda N)$. The objective is convex, so we use L-BFGS (tolerance 10^{-4}, at most 2000 iterations), and every run reaches the same optimum.
- *Random forest:* Gini impurity and bootstrap samples. Trees are grown greedily, so there is no gradient optimiser. Regularisation comes from `max_depth`, `min_samples_leaf` and `max_features`.
- *MLP:* ReLU hidden layers, a softmax output and the cross-entropy loss. We use Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$) with mini-batches of 32, a constant learning rate η and an L2 penalty α. Early stopping is done by tuning the number of epochs with grouped CV. We do not use scikit-learn's built-in early stopping, because its internal split is random by row and can put the same respondent on both sides.

Validation scheme. (1) *Tuning:* 5-fold **grouped** cross-validation (`GroupKFold` on respondent ID) on the 606 training rows. The `FeatureBuilder` is re-fitted inside every fold, so the vocabulary, medians and μ, σ never see the held-out fold. (2) *Model selection:* each family is refit on all training rows with its best hyperparameters and scored once on the 132 validation rows, and the highest validation macro-F1 wins. (3) *Final estimate:* the chosen model is trained on training + validation and scored once on the 132 test rows, which gives our single-number test estimate. Only after that do we retrain on all 870 rows for `pred.py`.

Hyperparameter search (Table 2). Scale parameters (C, α, η) are searched on log-spaced values. Each range goes from clear underfitting ($C = 10^{-3}$, depth 3) to clear overfitting ($C = 10^{2}$, unlimited depth). If the best value is at the edge of a range, we extend the range and search again. Softmax regression uses a full grid of 88 settings, and the forest and the MLP each use 60 random-search settings, so all three families get a similar budget. Settings are compared by mean grouped-CV macro-F1, with accuracy as the tie-breaker. If two settings are within one standard error, we pick the simpler one. To show that the ranges are reasonable, we will plot validation curves (CV score against C, `max_depth` and α) with the peak inside each range. **All three families are fully tuned before they are compared.** Tuning only the winner would be unfair to the others.

Table 2: Planned search, grouped 5-fold CV on the training split. Shared by all models: `min_df` $\in \{1, 2, 3, 5\}$, text weighting $\in$ {binary, TF-IDF}.

Model (search)	Hyperparameter	Values tried
Softmax regression (full grid, 88)	$C = 1/(\lambda N)$	11 log-spaced values, 10^{-3} to 10^{2}
Random forest (random, 60)	`n_estimators`; `max_depth`	$\{100, 300, 600\}$; $\{3, 5, 10, 20, \text{None}\}$
	`min_samples_leaf`; `max_features`	$\{1, 2, 5, 10\}$; $\{\sqrt{D},\ \log_2 D,\ 0.1D,\ 0.3D\}$
MLP (random, 60; 3 seeds each)	hidden layers; L2 penalty α	(32), (64), (128), $(64, 32)$; log-uniform $[10^{-5}, 1]$
	learning rate η	$\{10^{-4}, 3\times10^{-4}, 10^{-3}, 3\times10^{-3}\}$
	epochs (early stopping)	$\{25, 50, 100, 200, 400\}$

Evaluation metrics. For class k, $\mathrm{P}_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k+\mathrm{FP}_k}$, $\mathrm{R}_k = \frac{\mathrm{TP}_k}{\mathrm{TP}_k+\mathrm{FN}_k}$ and $\mathrm{F1}_k = \frac{2\mathrm{P}_k\mathrm{R}_k}{\mathrm{P}_k+\mathrm{R}_k}$.

- **Accuracy:** `pred.py` is graded on test accuracy, and our classes are balanced, so it is a meaningful headline number.
- **Macro-F1** $= \frac{1}{3}\sum_k \mathrm{F1}_k$ (the selection metric) weights every sport equally and drops sharply when a model neglects one class. For example, always predicting one class gives 33% accuracy but only 0.17 macro-F1. The TA-written test set may differ from ours in class mix and wording, so we want a model that does well on all three sports. We prefer macro- over weighted-averaging because no sport matters more than another.
- **Per-class precision/recall and the confusion matrix** show which sports get confused. We expect Basketball vs. Soccer, whose *ease* and *injury* distributions overlap (Fig. 1).
- **Mean ± SD over the 5 folds**, so that we do not over-interpret differences smaller than the fold-to-fold noise.

Preliminary check. To make sure the pipeline works, we ran each family with default settings through the same grouped CV on the training split (`preliminary_check.py`). These numbers are *untuned* and are not results.

Additional analyses. (i) A *feature-group ablation* that drops song, colour, food, feeling or the multi-select group in turn. TAs may name different songs, so we want to know how much we rely on a few strong words. (ii) A *soft-voting ensemble* of the three tuned models, evaluated with the same protocol. (iii) One-hot vs. ordinal Likert encoding, and word bigrams, both tested inside the same CV loop.

Table 3: Untuned grouped 5-fold CV on the training split (606 rows).

Model (default settings)	Accuracy (mean ± SD)	Macro-F1
Majority-class baseline	0.333 ± 0.000	0.167
Softmax regression ($C = 1$)	0.914 ± 0.022	0.914
Random forest (300 trees)	0.917 ± 0.024	0.917
MLP (64 hidden units, $\alpha = 10^{-3}$)	0.899 ± 0.013	0.900
kNN ($k = 15$)	0.837 ± 0.021	0.833

References

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.